

Q1. Describe your method

The design of this setup involves a smart phone with accelerometer capabilities (x,y,z). these values are streamed to Arduino cloud to a setup python device which collects them though a callback function. every callback saves the values into a deque which has a maxlen of 100 and is used when updating the graph. This creates a rolling window of new data.

Initially the problem faced is Dash will wait for a number of samples before its able to regenerate the graph which would give a choppy visual. To work around this, Patch() is used to create smoother display. An interval triggers every 200ms which reads the collected buffered data in each patch which causes the sliding incremented display over the redraw. I used a deque with maxlen to buffer the incoming accelerometer readings because it automatically discards the oldest data once the limit is reached.

Setup table

stage	what	Purpose
1	Phone collects and send data	Is the data source for the graph
2	IOT cloud -	Helps with syncing across the setup and transporting data
3	Python code	Buffers incoming values
4	Interval	Timer which triggers
5	Patch()	Updates graph with buffered values

Q2. Wrapper function

```
def streamGraph(app, graph_id, getSeries, SeriesName):
```

this allows any user to setup a smooth and live updating graph using Dash. It streams the data without replacing the whole graph each call and without the callback or buffering logic attached making it simpler to implement. They just need their own variables, this could be e.g temperature readings, a way to get their data and their graph.

The Parameters

App – Dash app

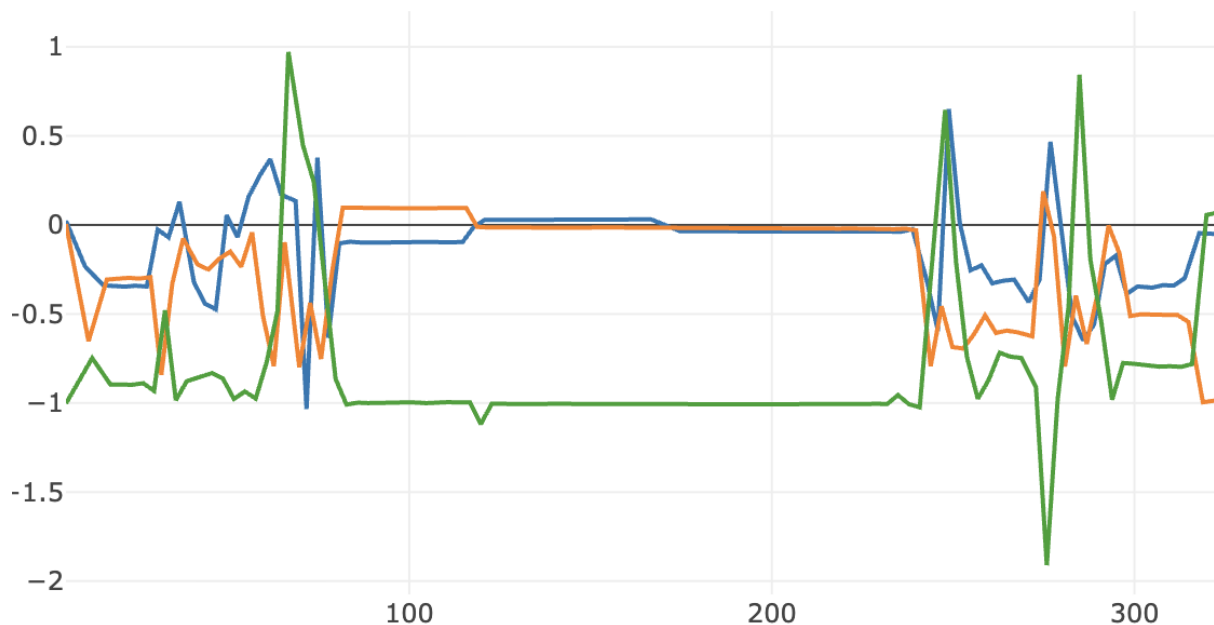
Graph_id – id of an existing graph to be displayed. Dcc.Graph has already designed the display of the graph.

getSeries - function which takes a series name and returns (x_values, y_values) for that series current data.

SeriesName – this is the list of names/variables to update like [x,y,z]

How the function works

The function sets a Dash callback which is based on the `dcc.Interval` tick setup. Each time the tick triggers it creates a `patch()` and fetches the most recent data on the variables with `getSeries()`. Which updates the `x,y,z` fields. with the use of `patch()` we don't have to return with a full graph each time, it just updates the dash values live so which creates a smoother display to track incoming numbers.



AI acknowledgement.

My assignment was created with the assistance of AI tool from Claude 2026 (anthropic). I used it to explain some concepts with code or understand how errors occurred. All outputs were assessed by me where I used it as a learning tool to bounce ideas from and explain areas.

Appendix 1.

explain how deque can be used when collectin data for my dash setup. it will add the new values and remove old ones along with `patch()` to update the values live

response:

A `deque` (double-ended queue) from Python's `collections` module is well suited to live data collection for a Dash dashboard because it can be initialized with a `maxlen` parameter, which creates a fixed-size rolling window. As new sensor readings arrive, they are appended to the deque; once the number of stored items exceeds `maxlen`, the oldest item is automatically discarded, so memory usage stays bounded even during long-running sessions.

In a Dash live-update setup, each incoming reading (paired with a timestamp) is appended to its corresponding deque as it arrives from the data source. Separately, on a fixed interval (e.g. via `dcc.Interval`), the current contents of the deque are read out and used to update the

graph. Rather than rebuilding the entire figure on each update, Dash's `Patch()` object can be used to modify only the changed parts of the existing figure — in this case, the `x` and `y` arrays of the relevant trace. This avoids the visible flicker or redraw that occurs when a full new figure is sent, since only the specific data fields are patched in place while the rest of the figure (layout, styling, other traces) remains untouched.

Together, the deque provides bounded, automatically-trimmed storage of recent values, while `Patch()` provides an efficient mechanism for pushing that data onto the graph incrementally — the combination allows a dashboard to display continuously updating data smoothly and efficiently over time.

Outcome: I was understanding how to create a rolling window affect with the data to have the graph include live values but keep the older ones aswell. Using deque along with `Patch()` was how I tried to implement this within the dash graph.

Appendix 2

Give a design for the app for dash, update the visuals and labels

Response:

```
app.layout = html.Div([
    html.H2("Live Accelerometer Stream", style={"textAlign": "center", "fontFamily":
"Arial"}),
    dcc.Graph(id="graph", figure={
        "data": [{"x": [], "y": [], "type": "scatter", "mode": "lines", "name": a} for a in Variables],
        "layout": {
            "title": "X / Y / Z Acceleration",
            "xaxis": {"title": "Time (s)"},
            "yaxis": {"title": "Acceleration (g)"},
            "plot_bgcolor": "#f9f9f9",
            "paper_bgcolor": "#f9f9f9",
        }
    }),
    dcc.Interval(id="tick", interval=200),
])
```

Outcome: I used this code as a template for my actual graph display, not all was kept as it was very overdone with things I didn't really need. I removed a lot of the extra styling and ended up using a black background dark mode setup instead.

Appendix 3.

Question: Explain how my callbacks work with arduino. I understand i setup the variables so when new values get pushed the callback saves them as data which i would use in the graph.

Response: You register three callbacks with `client.register("x", value=None, on_write=CallBack("x"))` (and same for y, z) — this tells Arduino IoT Cloud "watch these three cloud variables, and whenever any of them changes, call this function with the

new value." You don't call these callbacks yourself, and you don't ask Arduino for the current value at any point — Arduino Cloud calls your function automatically and hands it the new value the instant your phone's accelerometer sends fresh data through the cloud sync.

Inside the callback, all it does is store what it's given: `data[axis].append((time.time() - starttime, value))`. It timestamps the value (relative to when your script started) and appends it into that axis's deque. This is the entire job of the callback — receive and store, nothing more. It doesn't touch the graph, doesn't do any calculations, and doesn't know or care that a Dash app exists elsewhere in the script.

Outcome: Better understanding of how callbacks worked within my code and how it worked with Arduino cloud across the devices.